

BOOSTING MODERNO Y OPTIMIZACIÓN DE HIPER PARÁMETROS

20

26

01 ¿CÓMO APRENDE UN
MODELO BOOSTED?

02 BOOSTING

03 ¿CÓMO AJUSTAMOS SUS
HIPERPARÁMETROS SIN
HACER TRAMPA?

04 ¿CÓMO EVALUAMOS
PENSANDO EN EL
DESPLIEGUE?

05 ¿CÓMO EVALUAMOS
PENSANDO EN EL
DESPLIEGUE?

06 DEMO

TABLE OF CONTENTS

ENSEMBLE LEARNING

Crear un modelo de predicción de alta precisión es, sin duda, una tarea difícil. Nuestro principal reto es el ajustar el error de predicción, que se puede desglosar en:

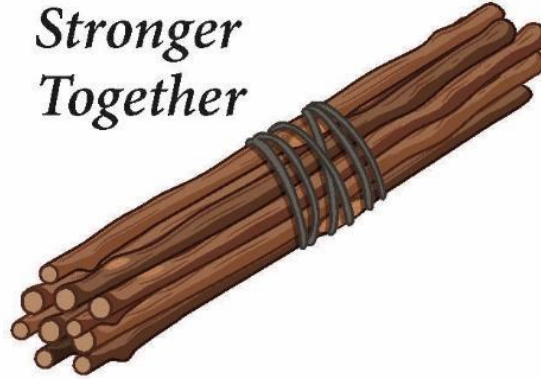
Bias: el promedio de la distancia entre la predicción y el valor objetivo.

Varianza: mide cuánto varían las predicciones de un modelo cuando se entrena con diferentes subconjuntos de los datos de entrenamiento. Muestra qué tan sensible es un modelo a pequeños cambios o al ruido en el conjunto de entrenamiento.

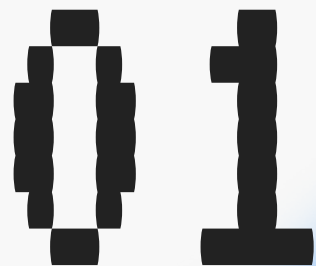
Ruido: es la variación aleatoria e impredecible o la perturbación no deseada en los datos que no representa el verdadero patrón subyacente. También se conoce como **«error irreducible»**, ya que, por muy complejo o perfecto que sea tu modelo de aprendizaje automático, el ruido nunca puede predecirse ni eliminarse mediante un modelo.

Ensemble Methods

*Stronger
Together*



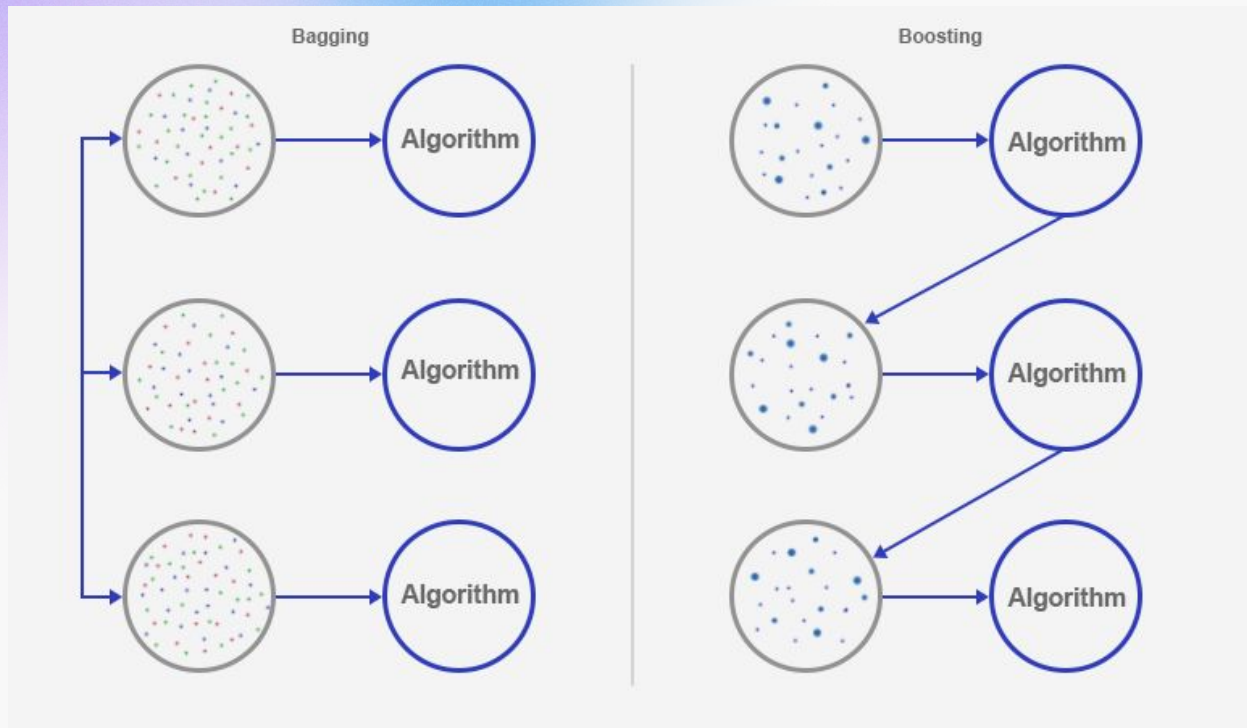
*Weak
Individually*



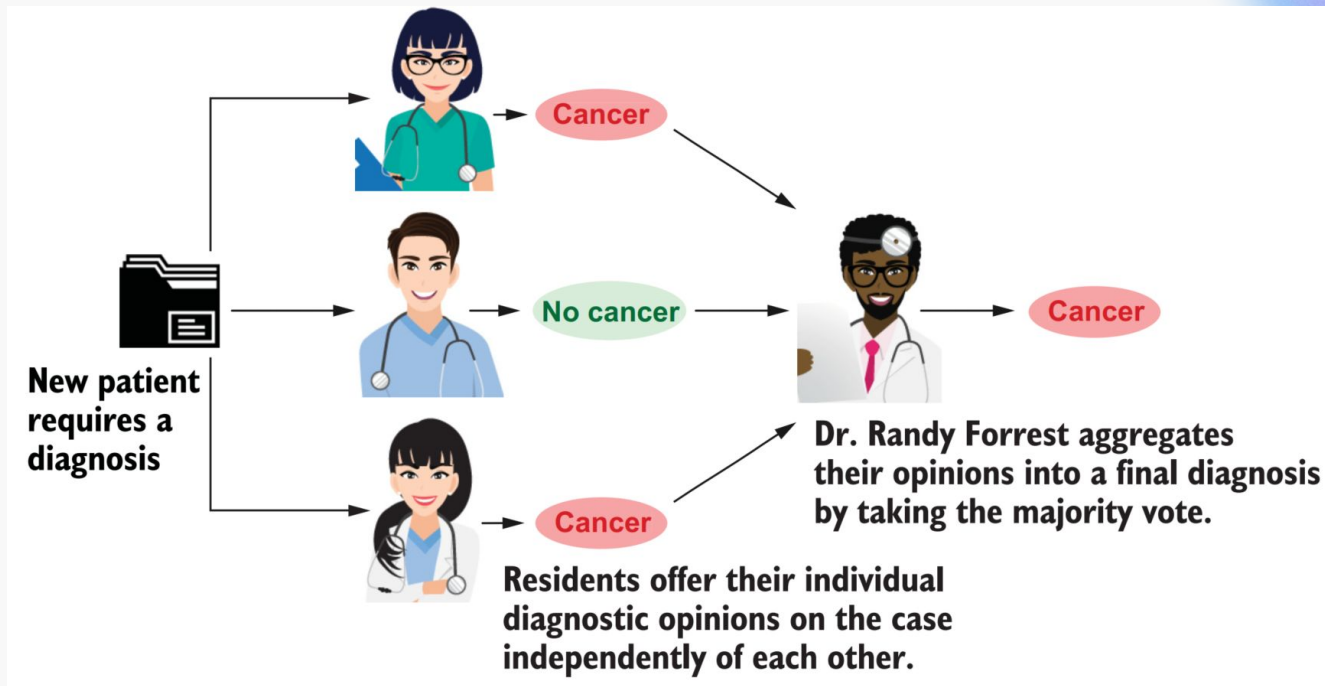
ENSEMBLE LEARNING

TÉCNICAS DE APRENDIZAJE POR ENSAMBLE

El aprendizaje por ensamble combinan las predicciones de varios modelos base (a menudo denominados «aprendices débiles») para generar una única predicción final más precisa y robusta. En lugar de basarse en un único modelo «óptimo», los métodos de conjunto promedian los errores individuales para reducir el sesgo, minimizar la varianza y evitar el sobreajuste.

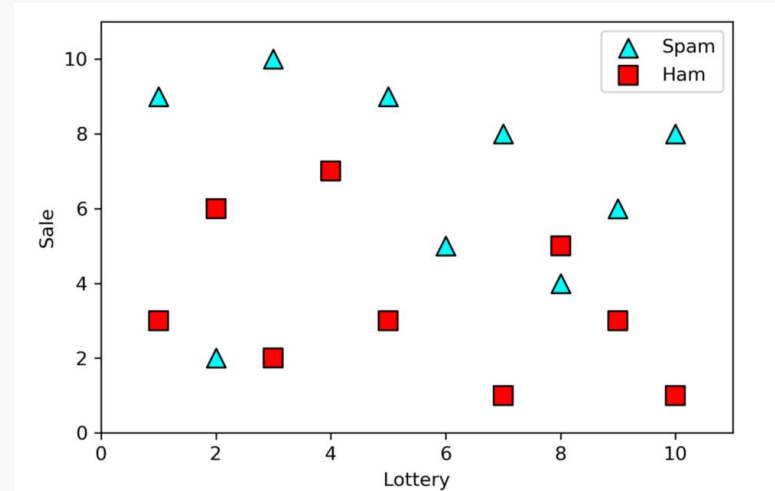


BAGGING

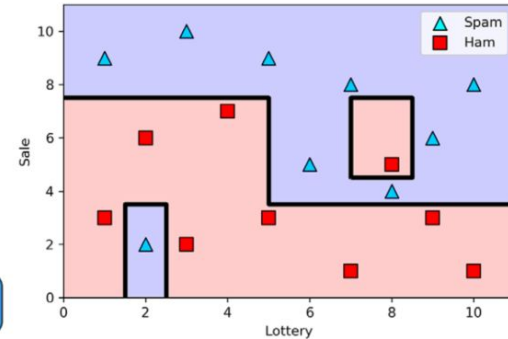
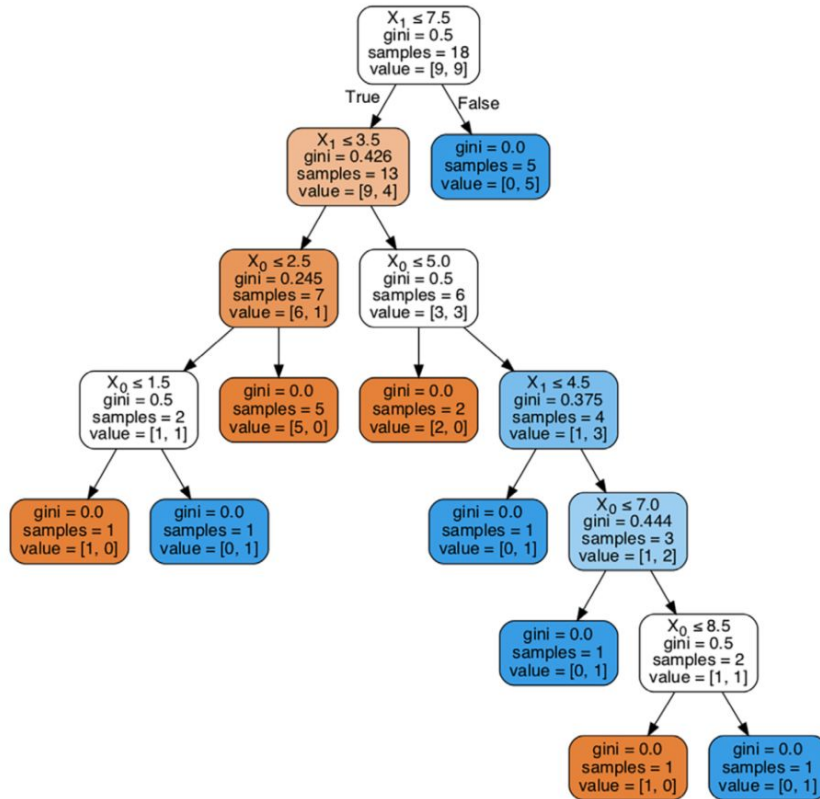


RANDOM FOREST

Lottery	Sale	Spam
7	8	1
3	2	0
8	4	1
2	6	0
6	5	1
9	6	1
8	5	0
7	1	0
1	9	1
4	7	0
1	3	0
3	10	1
2	2	1
9	3	0
5	3	0
10	1	0
5	9	1
10	8	1



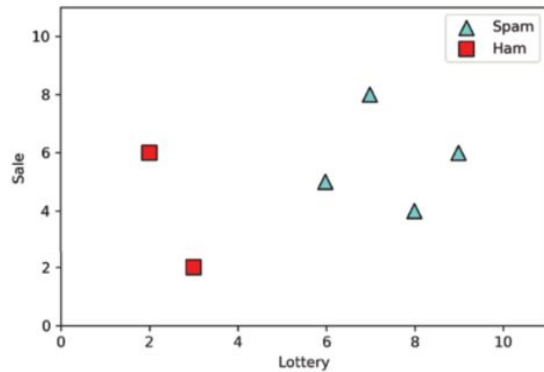
OVERFITTING UN ARBOL DE DECISION



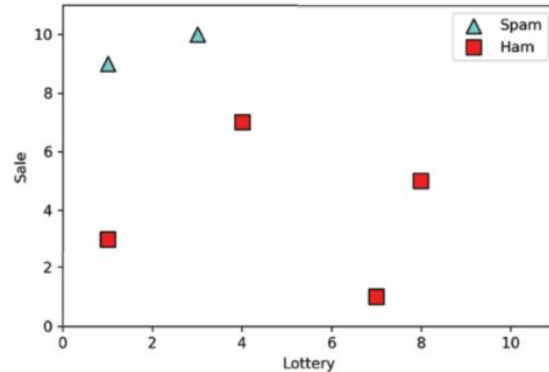
ENTRENAMIENTO DE UN RANDOM FOREST

Para este conjunto de datos, utilizamos tres modelos de aprendizaje débiles. Dado que el conjunto de datos tiene 18 puntos, consideremos tres subconjuntos de 6 puntos de datos

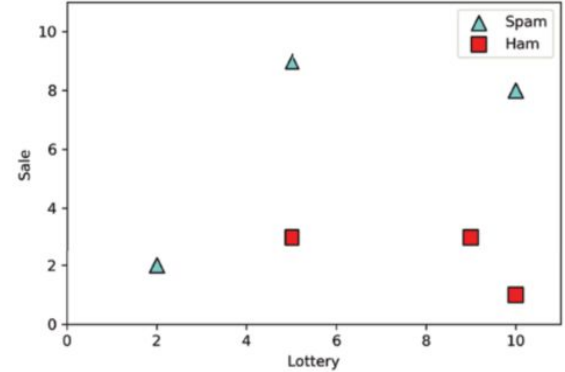
Subset 1



Subset 2



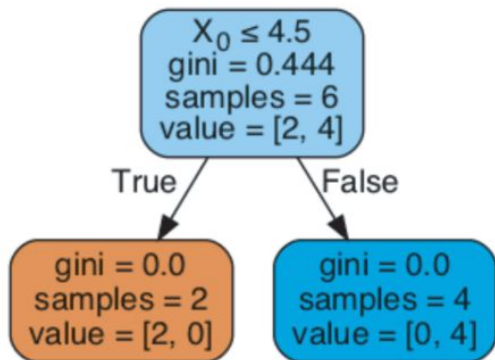
Subset 3



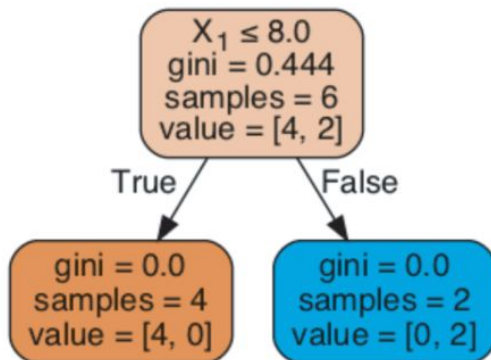
ENTRENAMIENTO DE UN RANDOM FOREST

Construimos nuestros tres modelos de aprendizaje débiles. Aplicamos un árbol de decisión de profundidad 1 a cada uno de estos subconjuntos. Un árbol de decisión de profundidad 1 contiene solo un nodo y dos hojas. Su límite consiste en una sola línea horizontal o vertical que divide el conjunto de datos de la mejor manera posible.

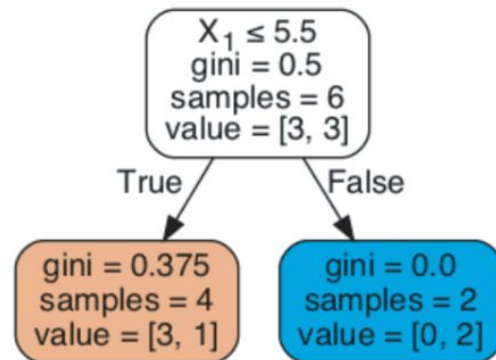
Weak learner 1



Weak learner 2



Weak learner 3

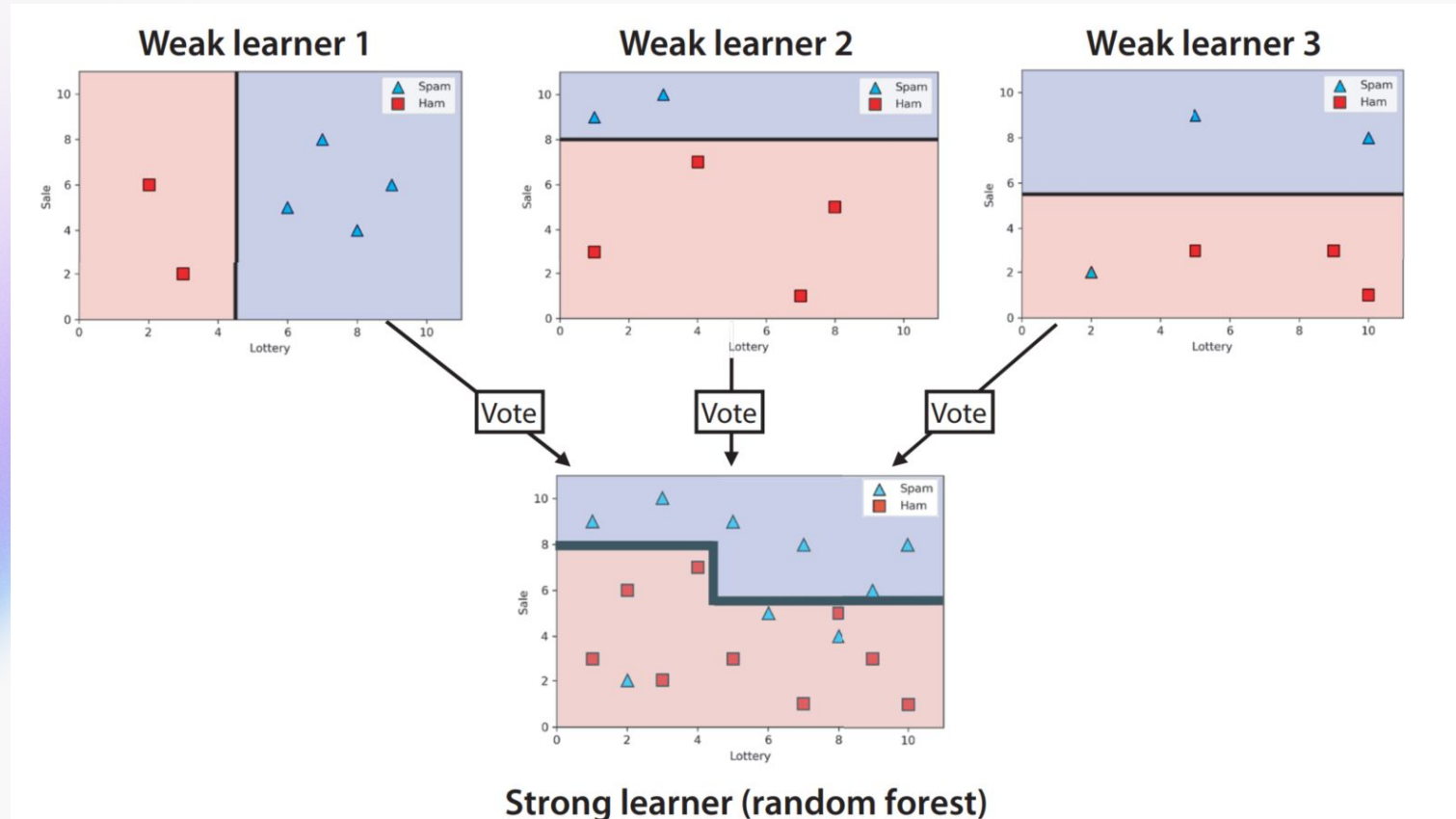


DECISION STUMP

Un árbol de
decisión de
profundidad 1 se le
conoce como
Decision Stump y lo
podemos traducir
como un muñon



ENTRENAMIENTO DE UN RANDOM FOREST



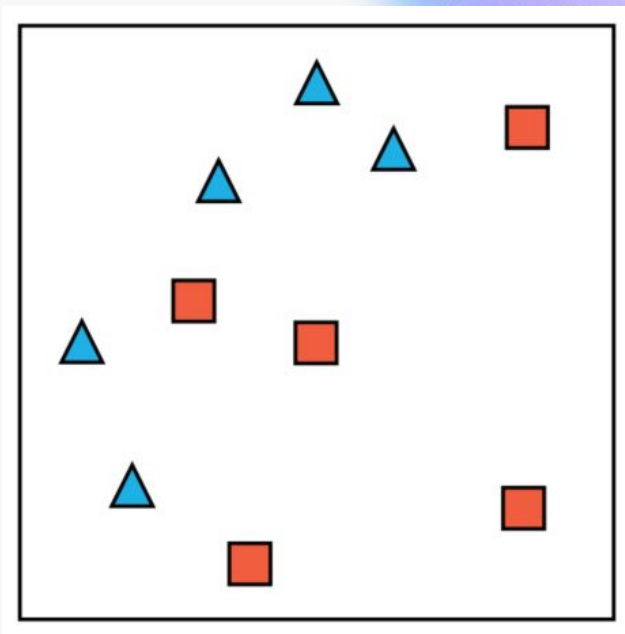
02

BOOSTING

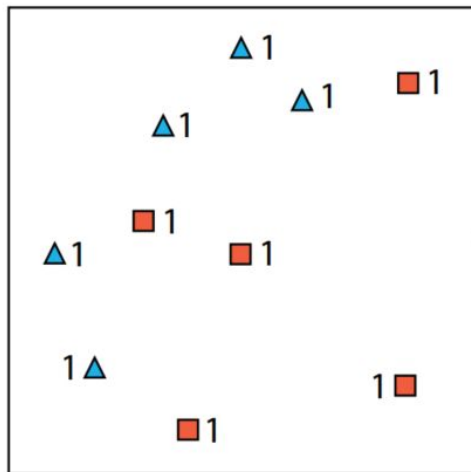
ADABOOST EL PRIMER ALGORITMO DE BOOSTING

Pseudocódigo para entrenar un modelo AdaBoost

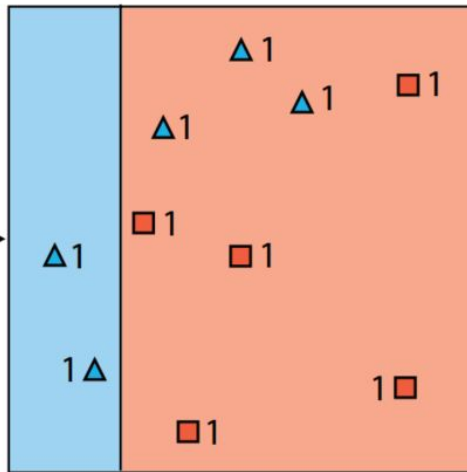
- Entrena el primer clasificador débil con el primer conjunto de datos.
- Repite el siguiente paso para cada nuevo clasificador débil:
 - Una vez entrenado un clasificador débil, los puntos se modifican de la siguiente manera:
 - Se amplían por medio de un peso los puntos que se clasificaron incorrectamente.
 - Entrena un nuevo clasificador débil con este conjunto de datos modificado.



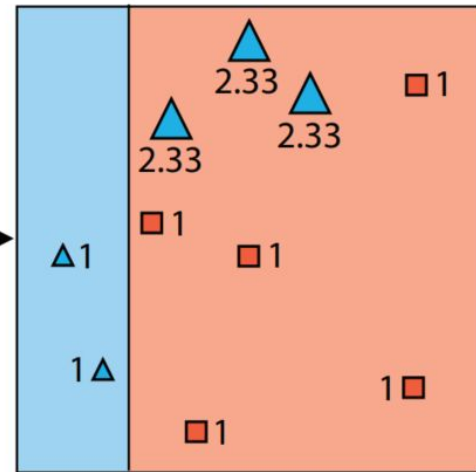
UNA VISIÓN GENERAL DE ADABOOST



Add a weight of 1
to every point.

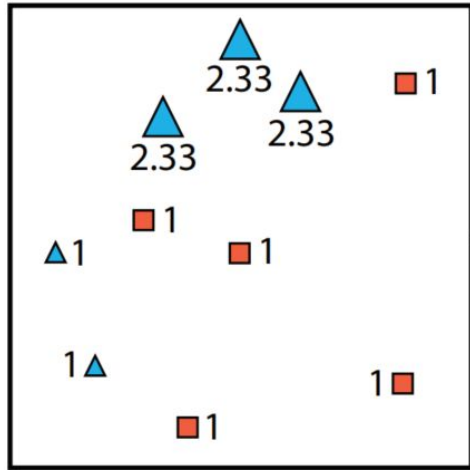


Fit a weak learner.
Correct: 7
Incorrect: 3

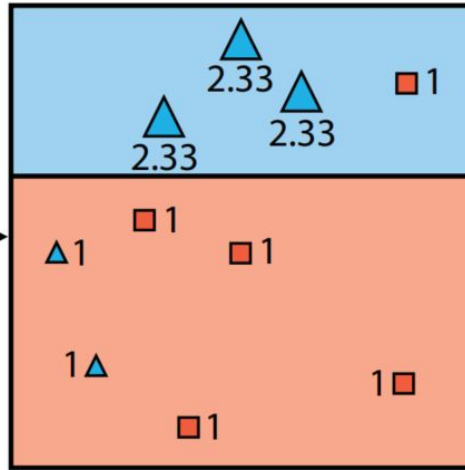


Rescale misclassified
points by $7/3$.

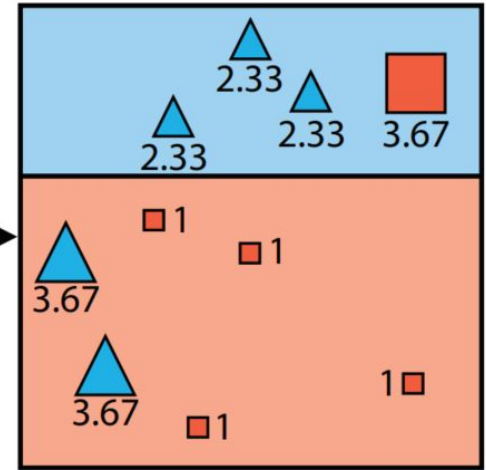
ENTRENAMOS EL SEGUNDO APRENDIZ DEBIL



The rescaled
dataset

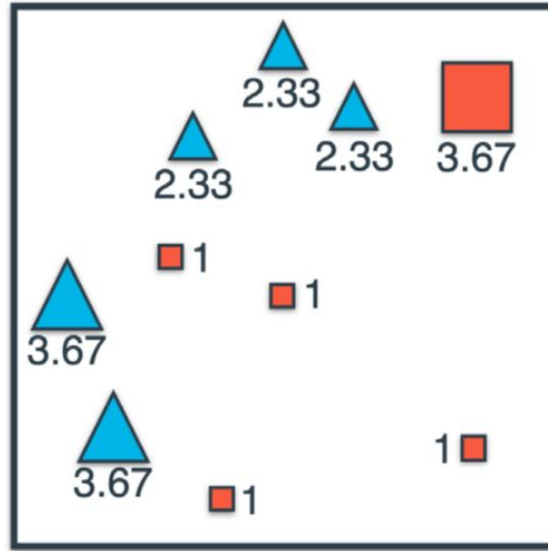


Fit a weak learner.
Sum of correct: 11
Sum of incorrect: 3

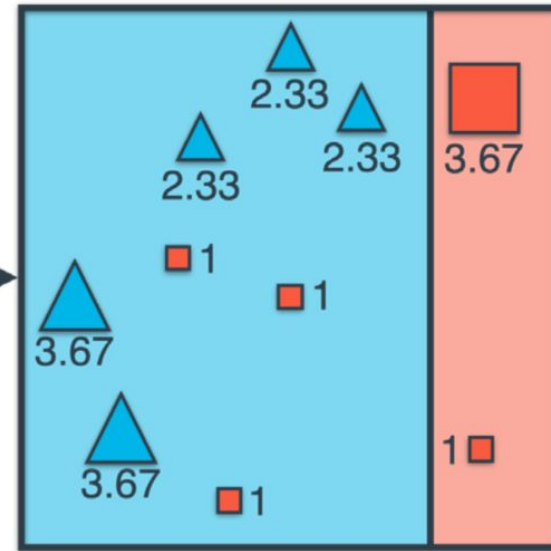


Rescale misclassified
points by $11/3$.

ENTRENAMIENTO DEL TERCER APRENDIZ DEBIL



The rescaled
dataset



Fit a weak learner
Sum of correct: 19
Sum of incorrect: 3

COMBINARLOS PARA FORMAR UN APRENDIZ FUERTE



APRENDIZ FUERTE

Voto ponderado

$$\begin{aligned} & (+2.2) \times (+1) \\ & + (0) \times (+1) \\ & + (-2.2) \times (-1) \\ \hline & = +4.4 \end{aligned}$$

Positivo → **SÍ**

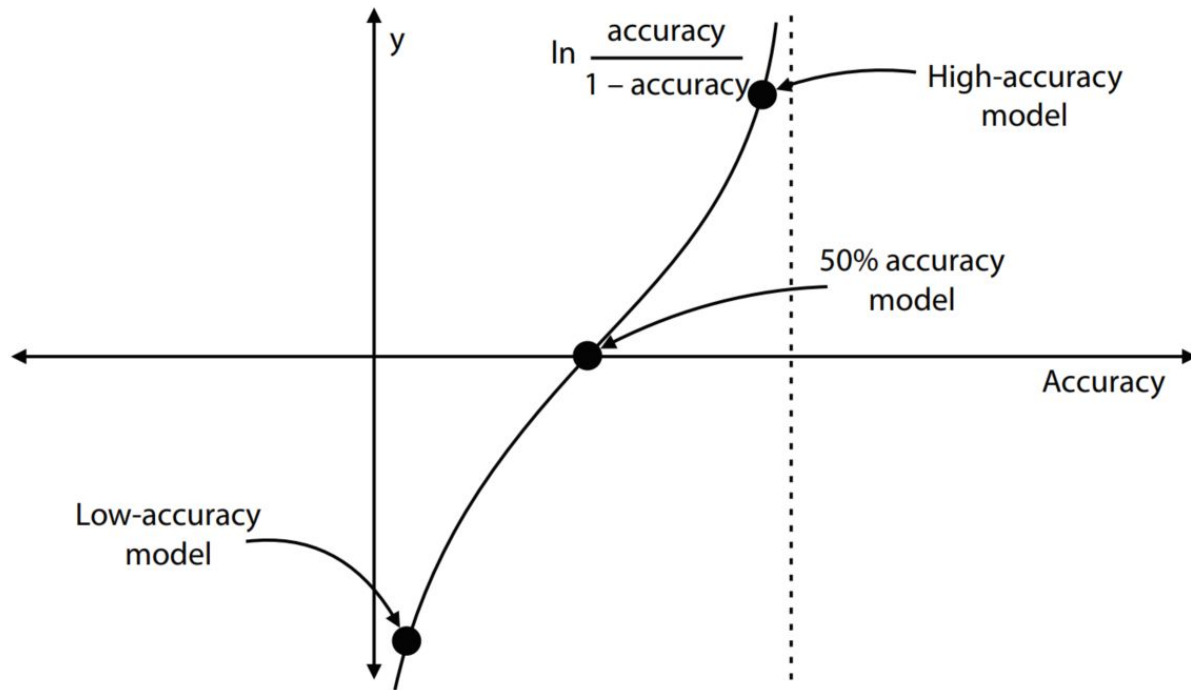
El "no" de Lenny cuenta como apoyo al "sí".

Peso del voto = $\ln(\text{exactitud} / (1 - \text{exactitud}))$ — positivo sobre 50%, cero en 50%, negativo por debajo.

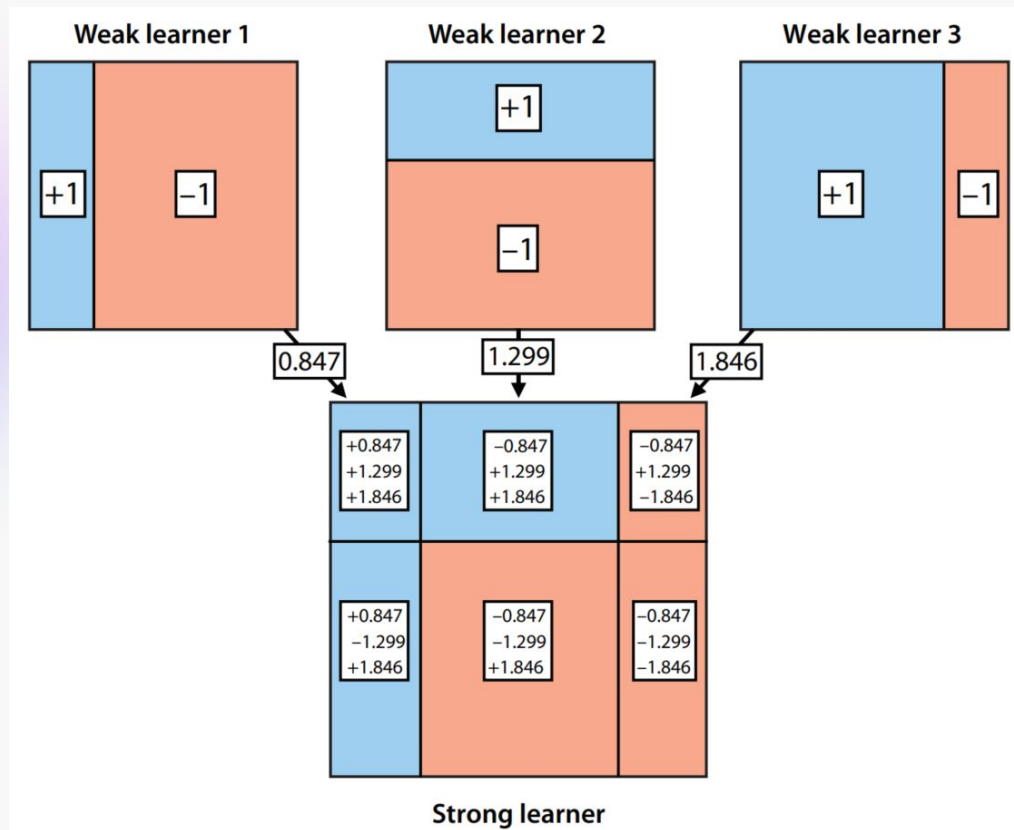
| = 50% de exactitud (lanzar una moneda)

PROBABILIDAD & FUNCIÓN LOG-ODDS

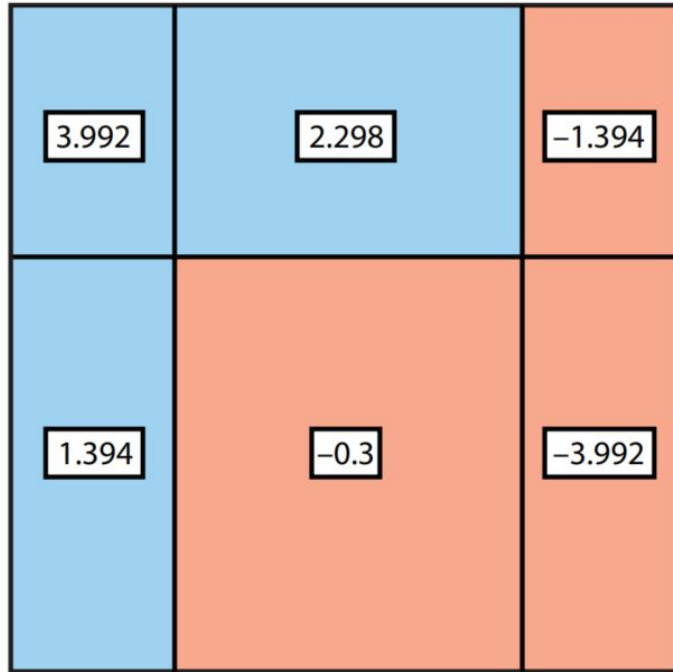
$$\text{log-odds}(x) = \ln \frac{x}{1-x}$$



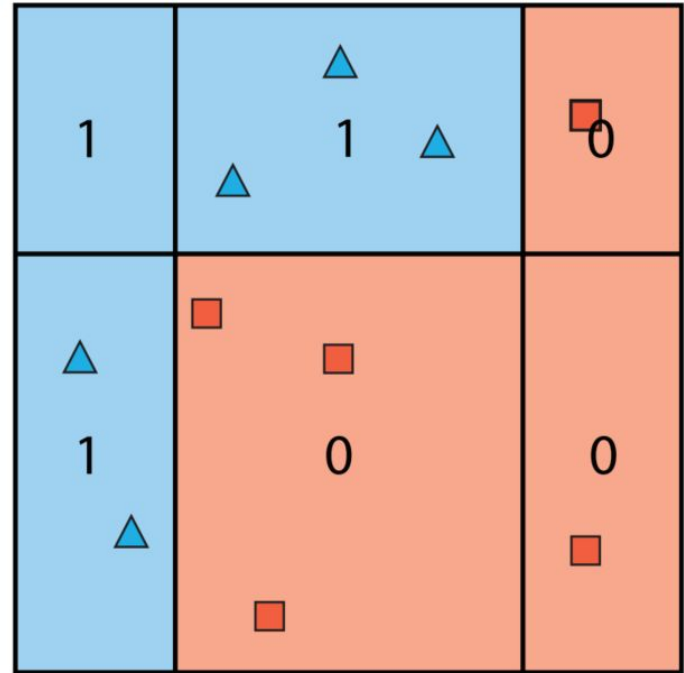
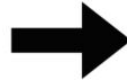
COMBINAMOS LOS APRENDICES DEBILES



COMBINAMOS LOS APRENDICES DEBILES



Votes

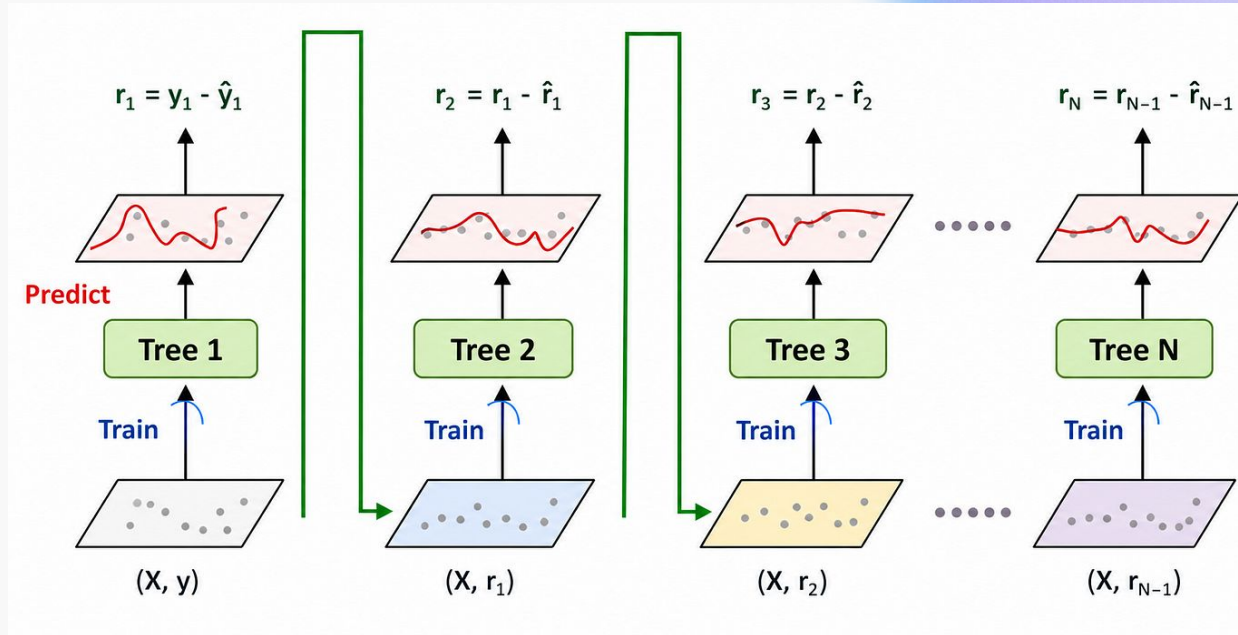


Predictions

GRADIENT BOOSTING

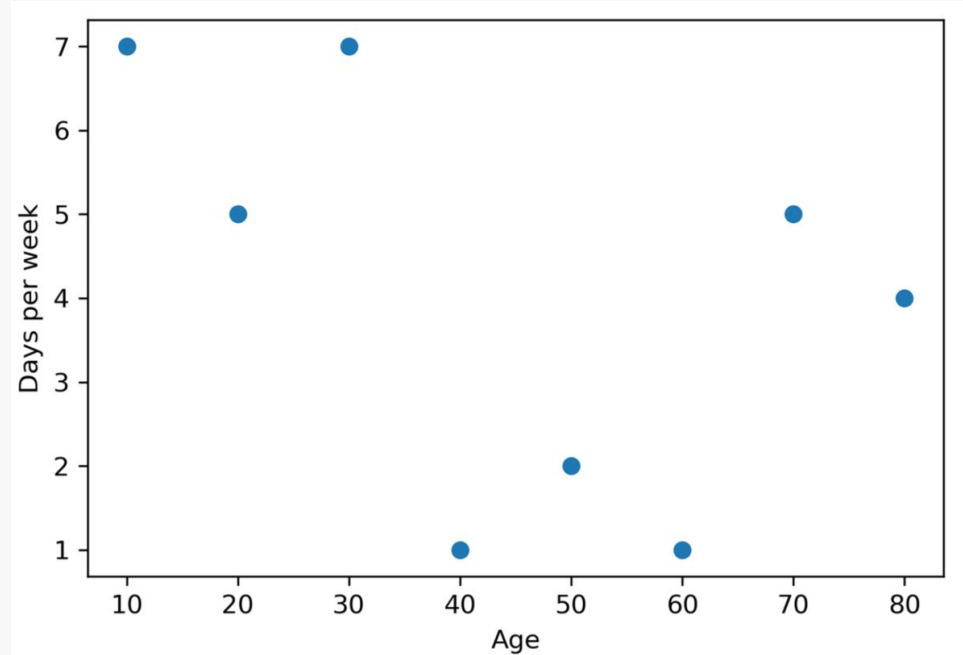
El gradient boosting es similar a AdaBoost, también usa árboles de decisión, y el objetivo de cada árbol es aprender de los errores de los anterior.

Una diferencia entre AdaBoost y el Gradient boosting es que este último permite árboles de decisión con una profundidad mayor que 1.



DATASET PARA GRADIENT BOOSTING

Feature (age)	Label (engagement)
10	7
20	5
30	7
40	1
50	2
60	1
70	5
80	4

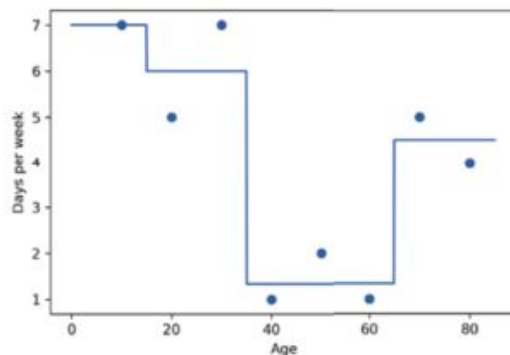
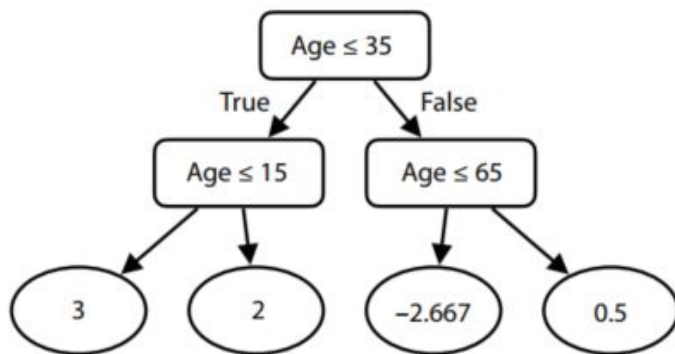


GRADIENT BOOSTING EL PRIMER APRENDIZ DÉBIL

Feature (age)	Label (engagement)	Prediction from weak learner 1
10	7	4
20	5	4
30	7	4
40	1	4
50	2	4
60	1	4
70	5	4
80	4	4

EL SEGUNDO APRENDIZ DÉBIL

Feature (age)	Label (engagement)	Prediction from weak learner 1	Residual	Prediction from weak learner 2
10	7	4	3	3
20	5	4	2	2
30	7	4	3	2
40	1	4	-3	-2.667
50	2	4	-2	-2.667
60	1	4	-3	-2.667
70	5	4	1	0.5
80	4	4	0	0.5



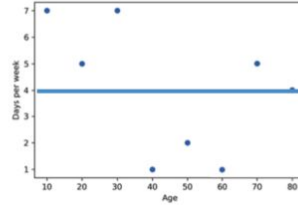
EL LEARNING RATE

Label	Prediction from weak learner 1	Prediction from weak learner 2	Prediction from weak learner 2 times the learning rate	Prediction from weak learners 1 and 2	Residual
7	4	3	2.4	6.4	0.6
5	4	2	1.6	5.6	-0.6
7	4	2	1.6	5.6	1.4
1	4	-2.667	-2.13	1.87	-0.87
2	4	-2.667	-2.13	1.87	0.13
1	4	-2.667	-2.13	1.87	-0.87
5	4	0.5	0.4	4.4	0.6
4	4	0.5	0.4	4.4	-0.4

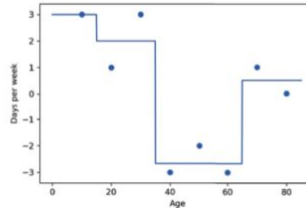
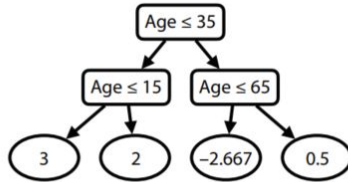
Y ASÍ SUCESIVAMENTE

Weak learner 1

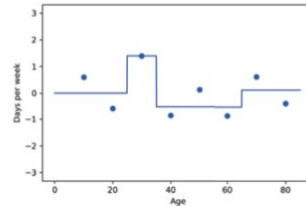
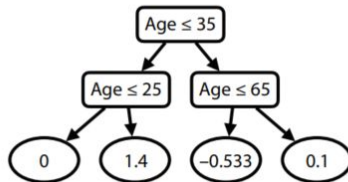
4



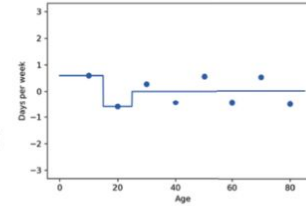
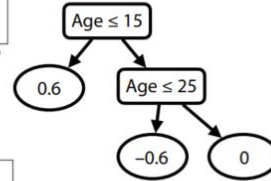
Weak learner 2



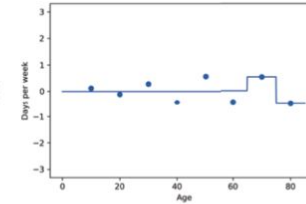
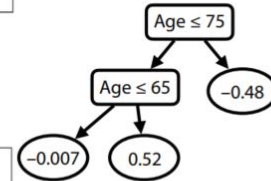
Weak learner 3



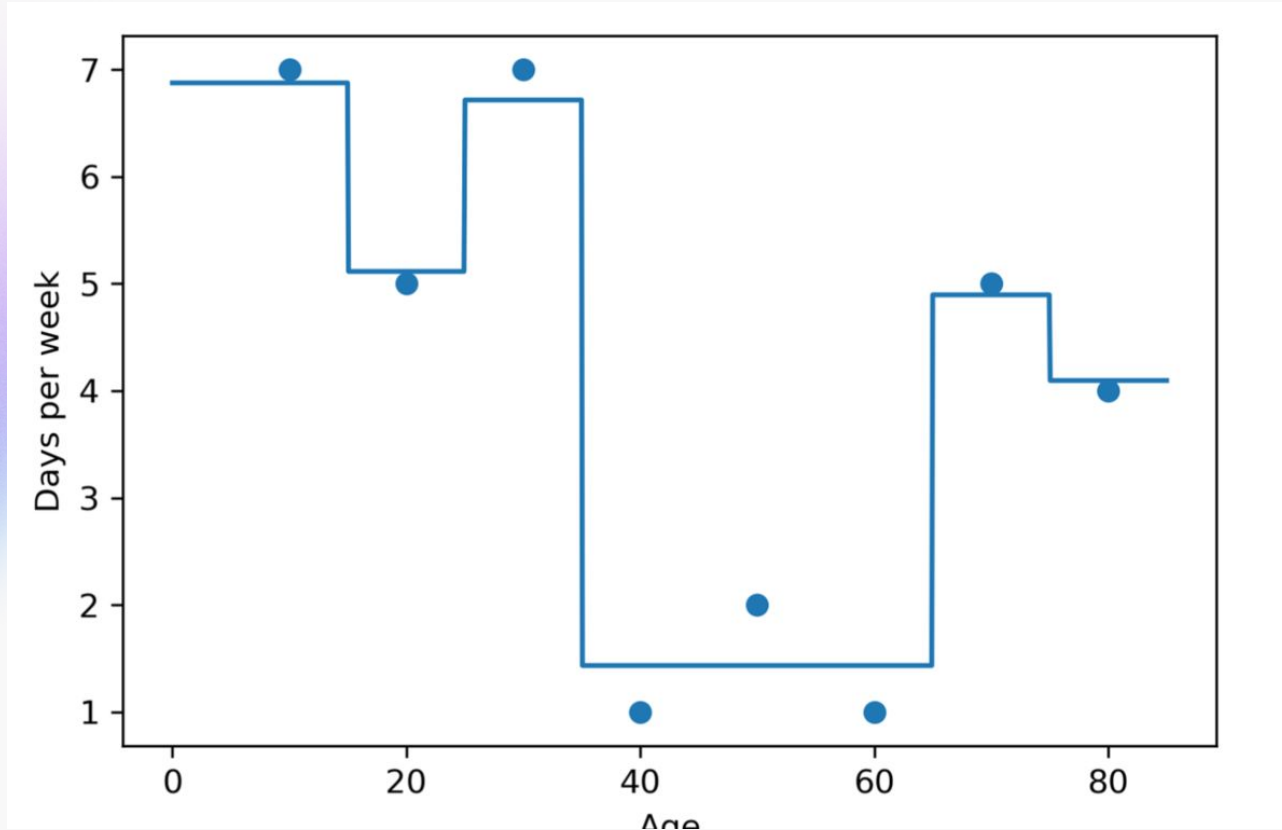
Weak learner 4



Weak learner 5

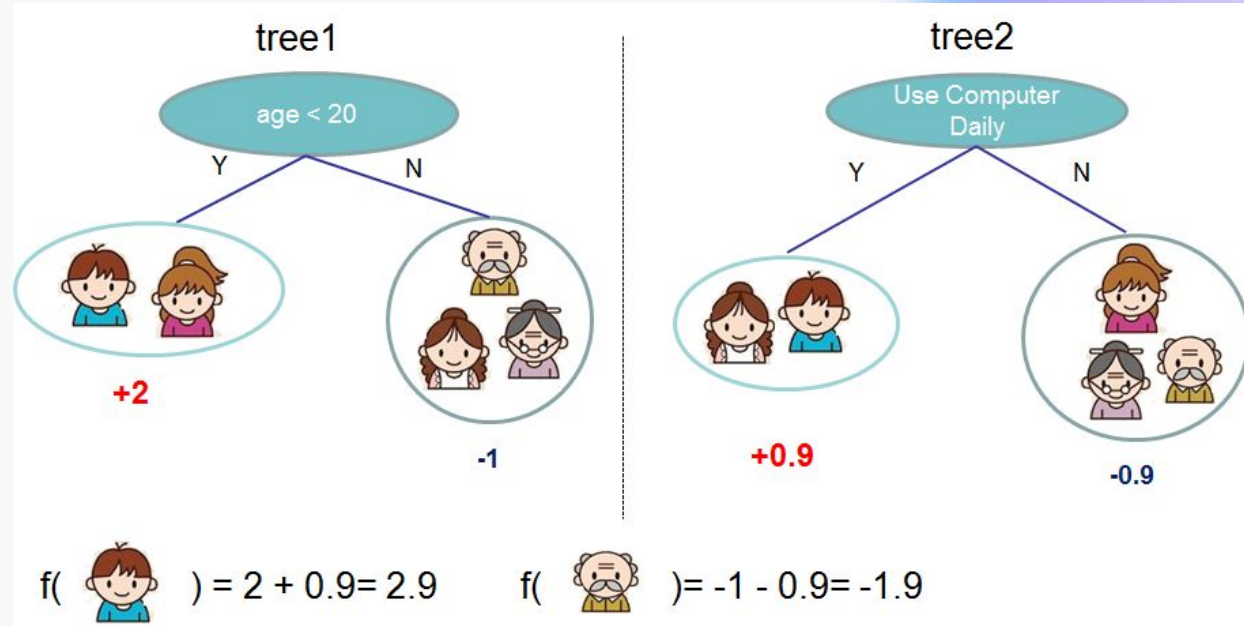


FINALMENTE EL APRENDIZ FUERTE



XGBOOST: GRADIENT BOOSTING AL EXTREMO

A diferencia del Gradient Boosting clásico, en XGBoost cuentas con dos mejoras clave. Por un lado, utilizas una métrica llamada puntaje de similitud para construir la estructura de los árboles de manera más eficiente. Por otro lado, incorporas un paso de poda para eliminar ramas innecesarias y evitar el sobreajuste.



XGBOOST SIMILARITY SCORE: UNA NUEVA Y EFECTIVA FORMA DE MEDIR LA SIMILITUD EN UN CONJUNTO

De los tres conjuntos siguientes, ¿cuál presenta mayor similitud y cuál menor?

- Conjunto 1: {10, -10, 4}
- Conjunto 2: {7, 7, 7}
- Conjunto 3: {7}

- **Set 1:** *Similarity score* = $\frac{(10 - 10 + 4)^2}{3} = 5.33$
- **Set 2:** *Similarity score* = $\frac{(7 + 7 + 7)^2}{3} = 147$
- **Set 3:** *Similarity score* = $\frac{7^2}{1} = 49$

Puntaje de similitud: Dado un conjunto $\{a_1, a_2, \dots, a_n\}$, el índice de similitud es el cuadrado de la suma de los elementos, dividido por el número de elementos:

$$\frac{(a_1 + a_2 + \dots + a_n)^2}{n + \lambda}$$

Existe un hiperparámetro λ asociado a la puntuación de similitud, que ayuda a evitar el sobreajuste.

DATASET PARA XGBOOST

Feature (age)	Label (engagement)	Prediction from the first weak learner	Residual
10	7	0.5	6.5
20	5	0.5	4.5
30	7	0.5	6.5
40	1	0.5	0.5
50	2	0.5	1.5
60	1	0.5	0.5
70	5	0.5	4.5
80	4	0.5	3.5

$$\text{Similarity} = \frac{(6.5 + 4.5 + 6.5 + 0.5 + 1.5 + 0.5 + 4.5 + 3.5)^2}{8} = 98$$

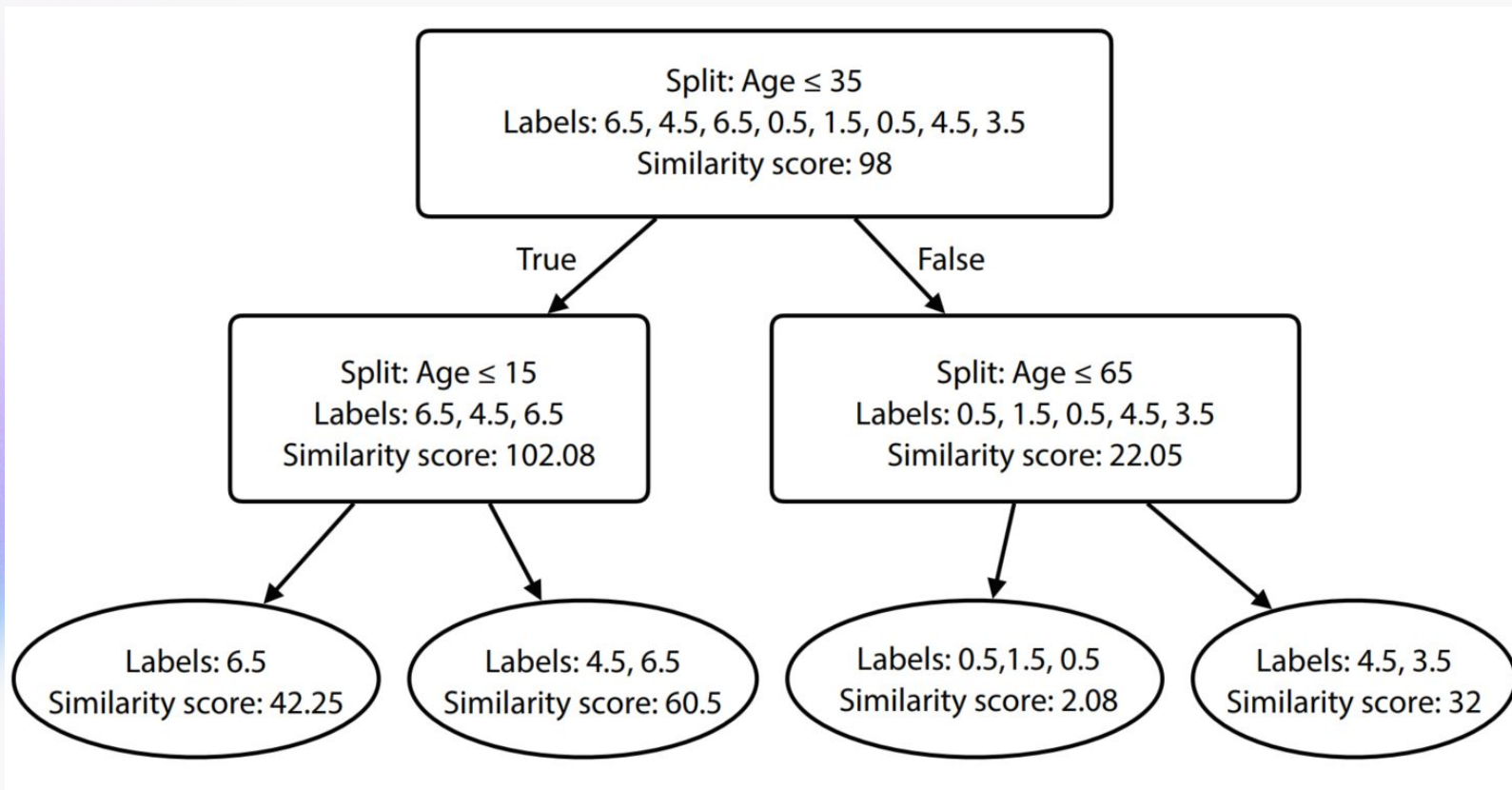
ELIGIENDO DONDE PARTIR EL PRIMER ARBOL

Feature (age)	Label (engagement)
10	7
20	5
30	7
40	1
50	2
60	1
70	5
80	4

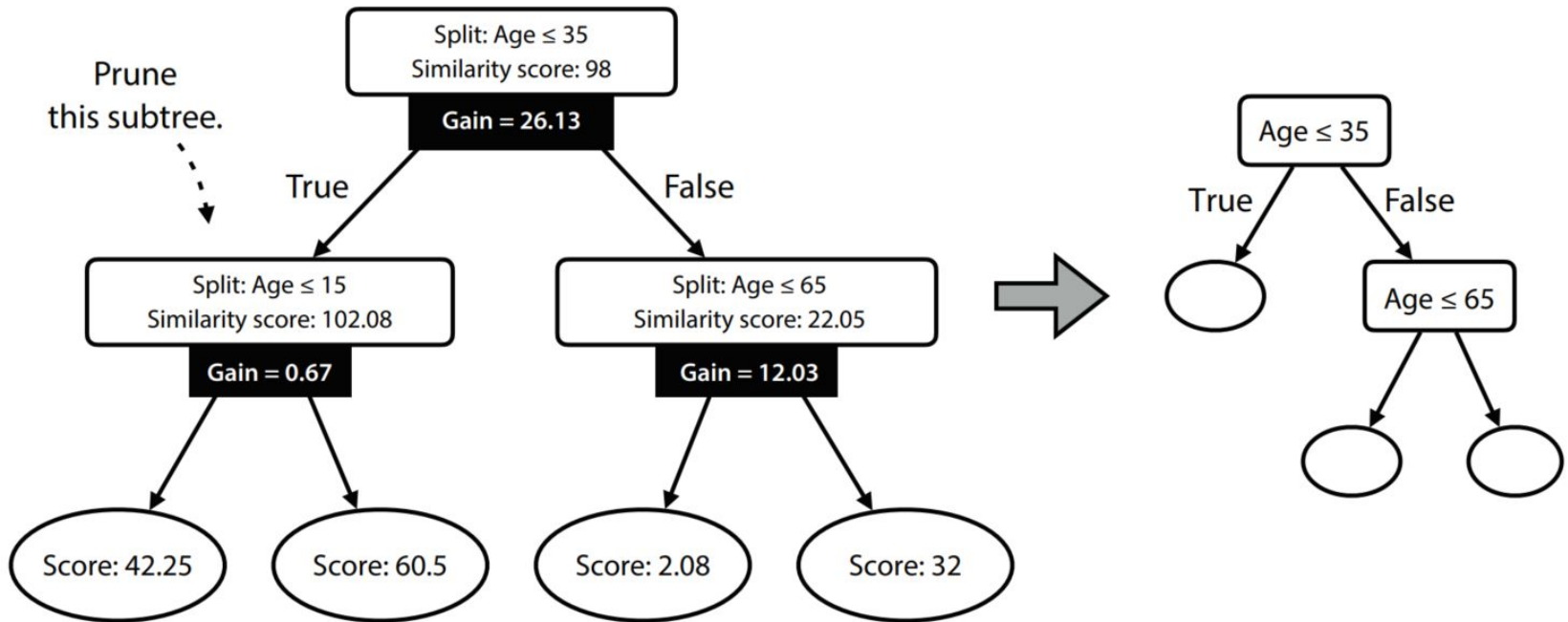
Split for the root node, with dataset {6.5, 4.5, 6.5, 0.5, 1.5, 0.5, 4.5, 3.5}, and similarity score = 98:

- Split at 15:
 - Left node: {6.5}; similarity score: 42.25
 - Right node: {4.5, 6.5, 0.5, 1.5, 0.5, 4.5, 3.5}; similarity score: 66.04
 - Combined similarity score: 108.29
- Split at 25:
 - Left node: {6.5, 4.5}; similarity score: 60.5
 - Right node: {6.5, 0.5, 1.5, 0.5, 4.5, 3.5}; similarity score: 48.17
 - Combined similarity score: 108.67
- Split at 35:
 - Left node: {6.5, 4.5, 6.5}; similarity score: 102.08
 - Right node: {0.5, 1.5, 0.5, 4.5, 3.5}; similarity score: 22.05
 - **Combined similarity score: 124.13**
- Split at 45:
 - Left node: {6.5, 4.5, 6.5, 0.5}; similarity score: 81
 - Right node: {1.5, 0.5, 4.5, 3.5}; similarity score: 25
 - Combined similarity score: 106
- Split at 55:
 - Left node: {6.5, 4.5, 6.5, 0.5, 1.5}; similarity score: 76.05
 - Right node: {0.5, 4.5, 3.5}; similarity score: 24.08
 - Combined similarity score: 100.13

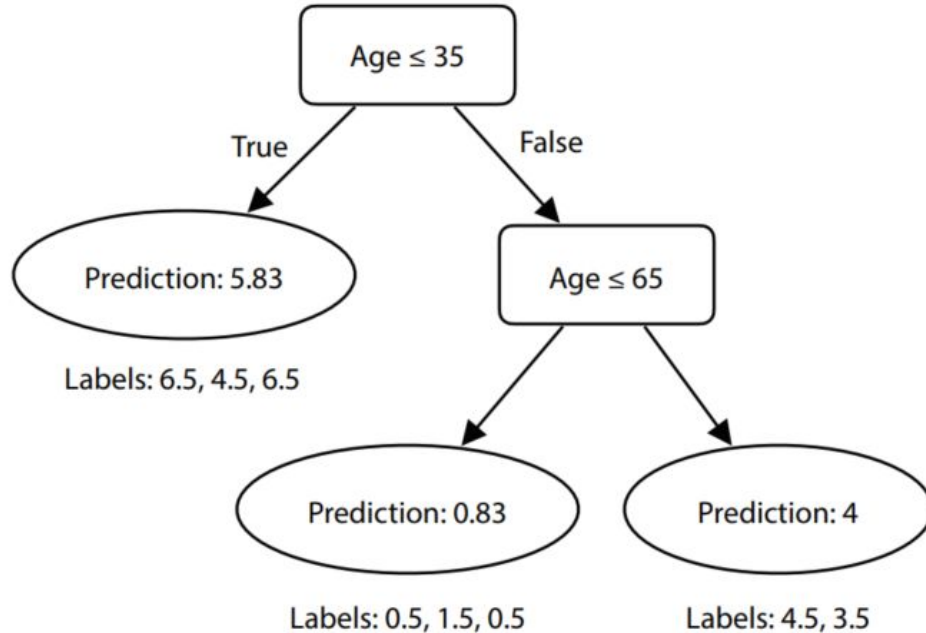
XGBOOST ENTRENANDO LOS OTROS APRENDICES



PODAR ÁRBOLES: UNA FORMA DE REDUCIR EL SOBREAJUSTE



¿CÓMO HACER LAS PREDICCIONES?



Obtenemos las predicciones de la misma manera que las obtenemos de cualquier árbol de decisión, es decir, calculando el promedio de las etiquetas en la hoja correspondiente

¿CÓMO HACER LAS PREDICCIONES?

Observe que las predicciones combinadas están más cerca de las etiquetas que las predicciones del primer aprendiz débil. El siguiente paso es iterar. Calculamos nuevos residuos para todos los puntos de datos, ajustamos un árbol a ellos, podamos el árbol, calculamos las nuevas predicciones combinadas y continuamos de esta manera. El número de árboles que queremos es otro hiperparámetro que podemos elegir al inicio.

Label (engagement)	Prediction from weak learner 1	Prediction from weak learner 2	Prediction from weak learner 2 times the learning rate	Combined prediction	Residual
7	0.5	5.83	4.08	4.58	2.42
5	0.5	5.83	4.08	4.58	0.42
7	0.5	5.83	4.08	4.58	2.42
1	0.5	0.83	0.58	1.08	-0.08
2	0.5	0.83	0.58	1.08	0.92
1	0.5	0.83	0.58	1.08	-0.08
5	0.5	4	2.8	3.3	1.7
4	0.5	4	2.8	3.3	0.7



DEMO

Tres implementaciones modernas de gradient boosting



XGBoost

2016

El estándar robusto

- Regularización explícita en la función objetivo (γ , λ , α)
- `tree_method="hist"` para velocidad
- Crece el árbol por niveles

Úsalo cuando: Primer candidato y baseline boosted defendible



LightGBM

2017 · Microsoft

La velocidad

- Histogramas + crecimiento por hojas
- Muestreo por gradiente (GOSS)
- Muy eficiente con millones de filas

Úsalo cuando: Datos grandes o muchas iteraciones de experimentación



CatBoost

2017 · Yandex

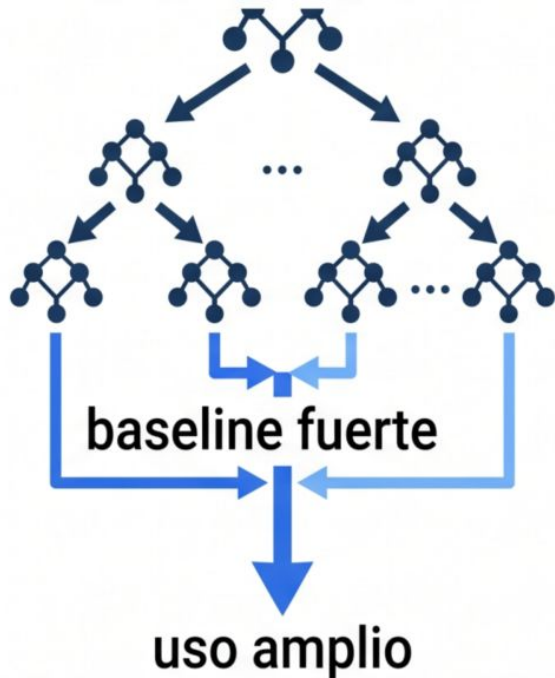
Las categóricas

- Codifica categorías sin fuga del target (ordered boosting)
- Árboles simétricos
- Buenos defaults

Úsalo cuando: Muchas variables categóricas relevantes

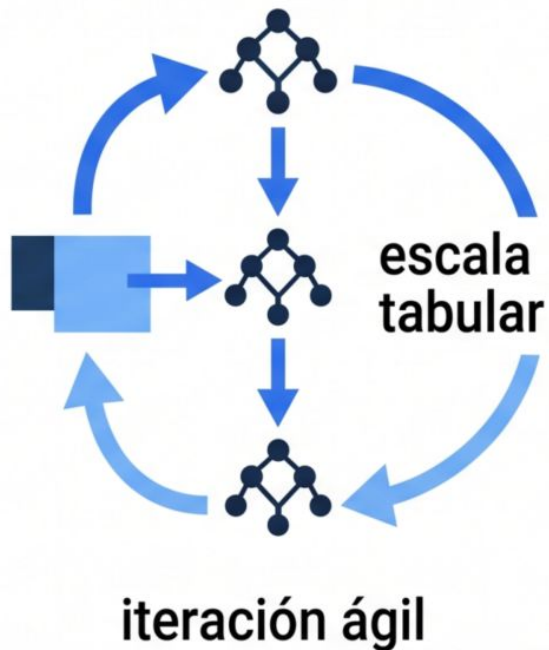
XGBoost

referencia madura



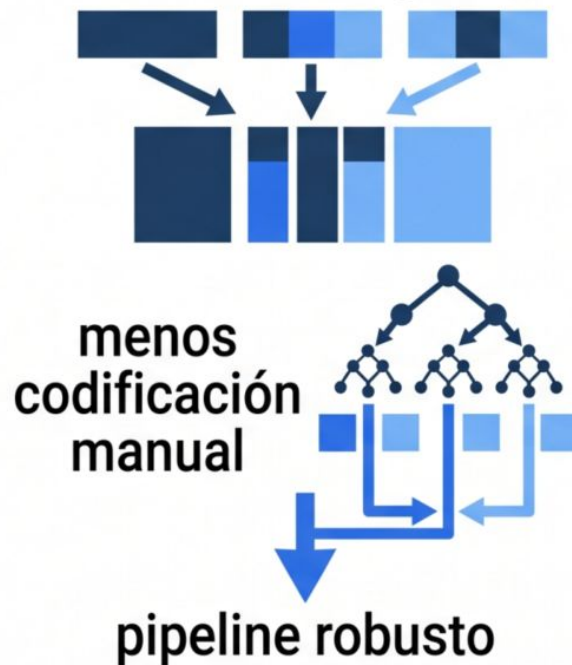
LightGBM

entrenamiento rápido



CatBoost

variables categóricas

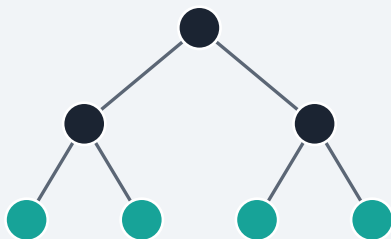


misma métrica + misma validación

¿Cómo crece cada árbol? Por niveles vs por hojas

Por niveles (level-wise)

XGBoost

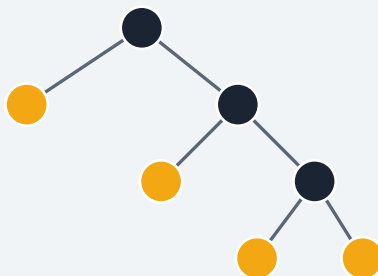


Llena un nivel completo antes de bajar.
Árboles balanceados.

Controlas con: `max_depth`

Por hojas (leaf-wise)

LightGBM

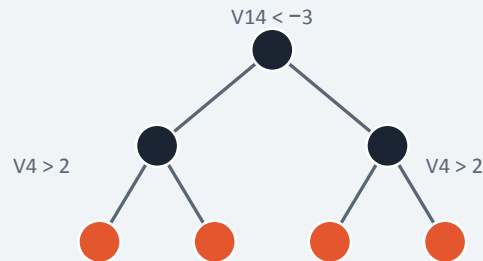


Siempre divide la hoja que más reduce el error. Árboles asimétricos y profundos donde importa.

Controlas con: `num_leaves`,
`min_child_samples`

Simétrico (oblivious)

CatBoost



La misma pregunta en todo un nivel.
Árboles muy regulares, rápidos al predecir.

Controlas con: `depth`

Regularización y subsampling: las perillas del modelo



Complejidad del árbol

`max_depth` · `num_leaves` · `depth`

↑ más interacciones entre variables, más riesgo de memorizar casos raros



Soporte mínimo por hoja

`min_child_weight` · `min_child_samples`

↑ más conservador: no crea hojas con muy pocos casos



Penalización

`reg_lambda` (L2) · `reg_alpha` (L1) · `gamma`

↑ más conservador: encoge las hojas y exige ganancia mínima para dividir

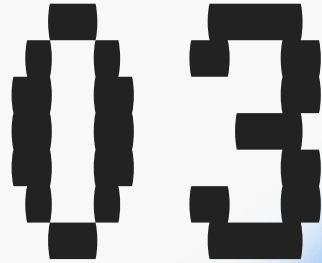


Aleatoriedad (subsampling)

`subsample` (filas) · `colsample_bytree` (columnas)

< 1: cada árbol ve una parte distinta de los datos → menos varianza

Ninguna perilla "mejora" por definición: cada una es una hipótesis que la validación debe confirmar.



¿CÓMO AJUSTAMOS SUS
HIPERPARÁMETROS SIN
HACER TRAMPA?

¿Qué son los Hiperparámetros?

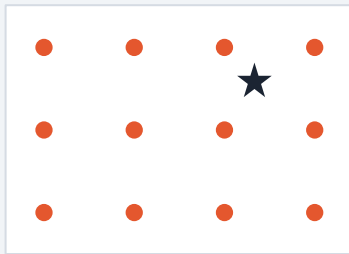
La Analogía del Café Perfecto

-  **Preparación Física:** Tienes "perillas" como la temperatura del agua, el gramaje y el tiempo de extracción.
-  **En Machine Learning:** Estas perillas son la tasa de aprendizaje, la profundidad del árbol o los nodos de la red.
-  **El Objetivo Central:** Encontrar la combinación exacta que minimice el error de predicción del modelo.



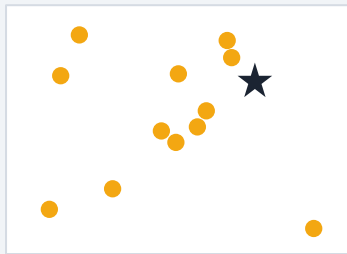
Buscar hiperparámetros: tres estrategias

Parámetros: los aprende el modelo (splits, valores de las hojas). **Hiperparámetros:** los fijamos antes de entrenar (learning_rate, max_depth...).



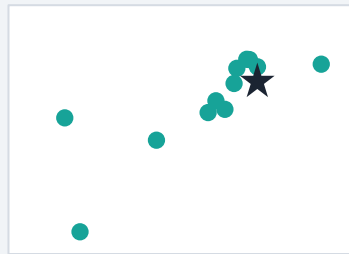
Grid search

Prueba todas las combinaciones de una grilla. Con 9 hiperparámetros $\times$ 5 valores: ~2 millones de modelos.



Random search

Prueba combinaciones al azar. Cubre mejor el espacio con el mismo presupuesto.



Optimización bayesiana

Aprende de los intentos anteriores y concentra la búsqueda donde parece haber buenos resultados.

★ = mejor configuración (desconocida) · cada punto = un modelo entrenado y validado

El Problema de la Búsqueda Ciega

Históricamente se usaban dos métodos sin "memoria" para buscar combinaciones.



Grid Search

- **Estrategia:** Probar absolutamente todas las combinaciones en una grilla.
- **Ventaja:** Es un método exhaustivo que no deja nada por fuera.
- **Problema:** Computacionalmente inviable. Nueve hiperparámetros generan millones de modelos.



Random Search

- **Estrategia:** Probar combinaciones generadas de forma completamente aleatoria.
- **Ventaja:** Cubre mucho mejor el espacio de búsqueda con el mismo presupuesto.
- **Problema:** Depende de la suerte y no aprende de los intentos desastrosos.

Optimización Bayesiana: Buscar con Inteligencia

Soluciona el problema aprendiendo de la experiencia pasada para hacer suposiciones educadas.



El Mapa Mental (Modelo Sustituto)

Cada vez que el sistema prueba hiperparámetros, anota el resultado. Con el tiempo, dibuja un mapa probabilístico que indica qué zonas prometen mejores resultados.



Explotación

Decide ir a la zona del mapa que ya sabemos que da buenos resultados, ajustando sutilmente las perillas para intentar exprimir un rendimiento aún mayor.



Exploración

Decide probar una zona del mapa completamente nueva y desconocida. Podría fallar, pero asegura no ignorar la configuración perfecta.

OPTUNA: EL ASISTENTE AUTOMATIZADO

Librería de Python diseñada para automatizar la búsqueda optimizada de hiperparámetros.



OPTUNA

Define-by-Run

Permite definir el espacio de búsqueda de forma dinámica utilizando bucles y condicionales nativos de Python.

Compatibilidad Universal

Se integra perfectamente con PyTorch, TensorFlow, Scikit-Learn, XGBoost, LightGBM y CatBoost.

Algoritmo TPE por Defecto

Utiliza el Estimador de Parzen Estructurado en Árbol (TPE) para guiar la búsqueda inteligente rápidamente.

Poda Inteligente (Pruning)

Detiene tempranamente los entrenamientos con mal rendimiento para ahorrar masivas cantidades de tiempo de cómputo.

¿Cómo decide Optuna? El Algoritmo TPE

TPE (Tree-structured Parzen Estimator) divide el historial en grupos para tomar atajos.

1. La División

Tras varios intentos, ordena los resultados. Toma el top 10% y los clasifica como "Buenos". El 90% restante va al grupo "Malos".

2. Los Perfiles

Modela la distribución estadística de cada grupo. Analiza qué valores compartían los intentos exitosos y cuáles los fracasos.

3. La Selección

No elige al azar. Sugiere el siguiente valor evaluando dónde la probabilidad del grupo "Bueno" supera por más margen a la del "Malo".

Por qué TPE es el estándar de la industria:

- **Excelente con categorías:** Maneja perillas de texto (ej. "Adam" vs "SGD") separándolas igual.
- **Velocidad:** Calcular dos curvas de probabilidad es computacionalmente mucho más barato.
- **Maneja la irrelevancia:** Si un parámetro no afecta, ambos perfiles serán iguales y TPE deja de ajustarlo.

Poda Inteligente (Pruning)

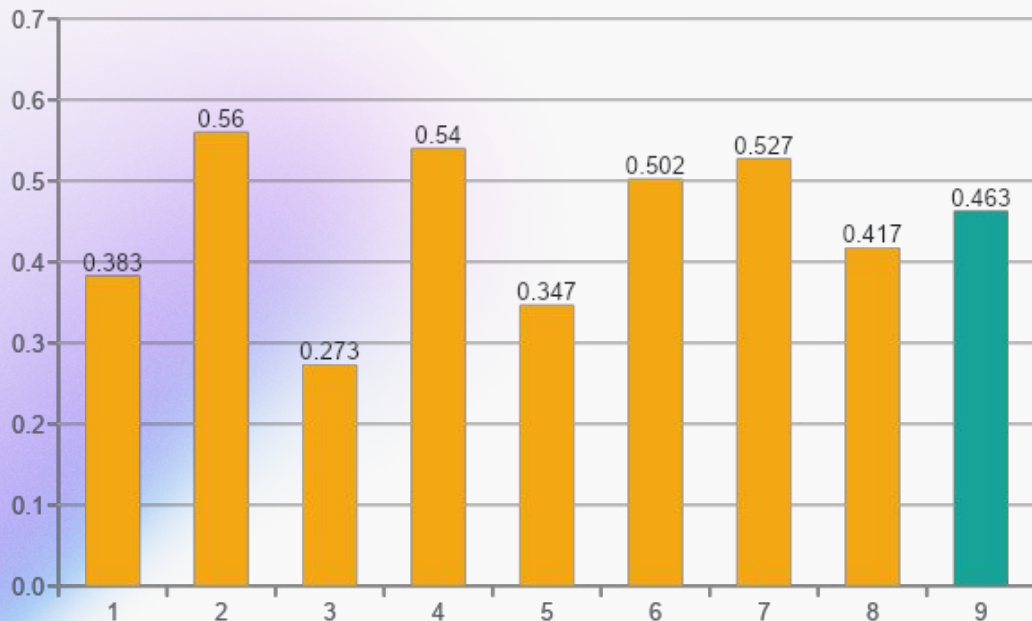
Cortar el fracaso de raíz.

-  **El Problema:** Entrenar modelos completos (especialmente Boosting o Deep Learning) cuesta tiempo.
-  **La Solución:** Optuna evalúa el rendimiento del modelo paso a paso mientras se entrena.
-  **La Acción:** Si el modelo va peor que la mediana de intentos anteriores, el "trial" se cancela inmediatamente.
-  **El Beneficio:** Evita perder horas en configuraciones desastrosas, liberando recursos para pruebas viables.



CV primero, optimizador después

Mismo modelo, mismos hiperparámetros: PR-AUC según el split



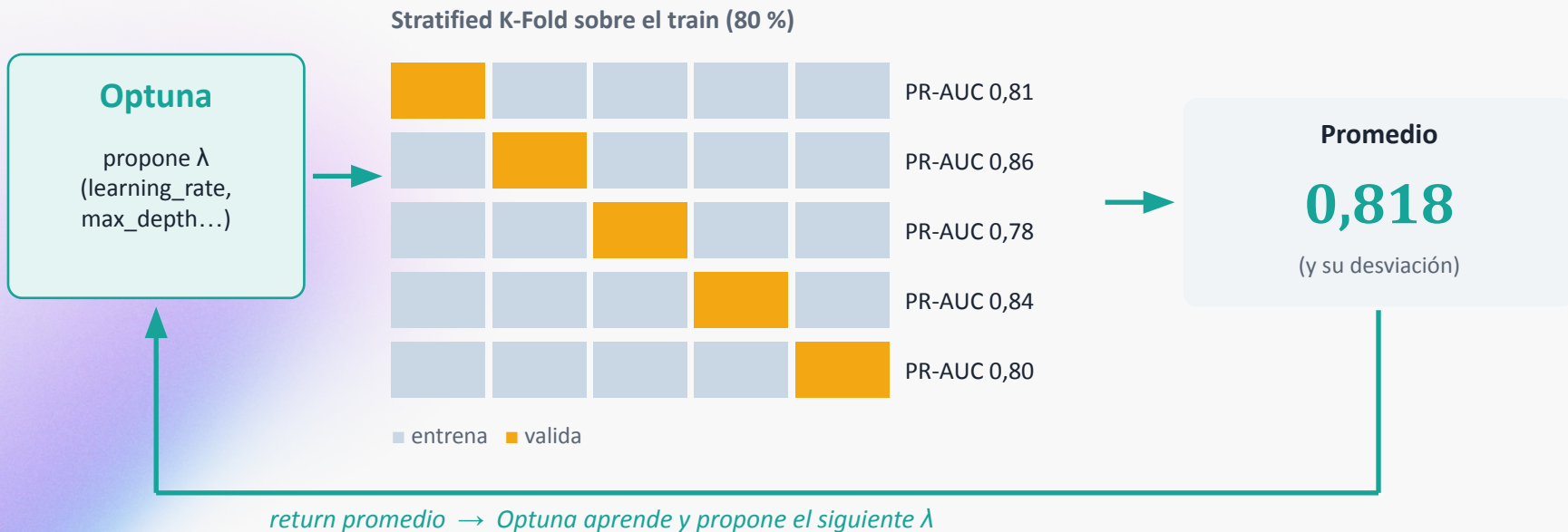
Simulación: 30 000 filas con 0,6 % de positivos, un solo modelo evaluado con 8 splits distintos. CV: 0.46 ± 0.06 .

De 0.27 a 0.56

solo por cambiar qué filas quedaron en validación.

- Si el score de cada trial es así de ruidoso, Optuna optimiza la suerte, no el modelo.
- Por eso: primero se fija la métrica y el esquema de CV; recién entonces se optimiza.
- Aun así, el mejor trial es optimista ("sesgo del ganador"): se re-evalúa después.

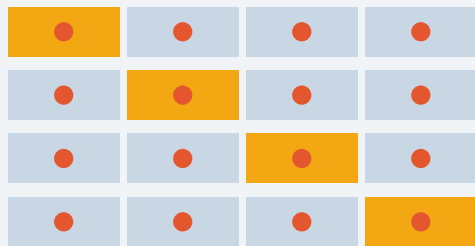
La validación cruzada vive dentro de cada trial



Test (20 %): no aparece en ningún trial

¿Qué tipo de folds? Depende de los datos

Stratified K-Fold

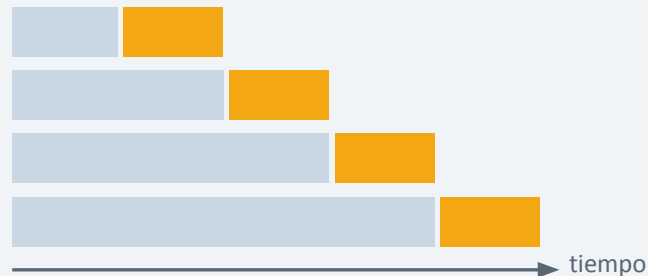


● = fraudes: misma proporción en cada fold

- Filas mezcladas al azar, sin orden temporal
- Cada fold tiene $\approx 0,17\%$ de fraude

Úsalo si: las observaciones son independientes del tiempo

Validación temporal (TimeSeriesSplit)

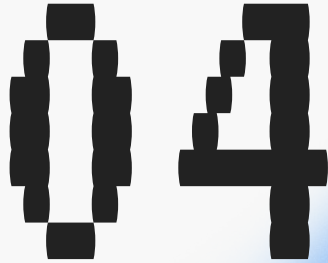


- Siempre entrena con el pasado y valida con el futuro
- Nunca mezclar al azar

Úsalo si: el modelo se usará sobre datos futuros (fraude en producción, demanda, churn)



DEMO



¿CÓMO EVALUAMOS
PENSANDO EN EL
DESPLIEGUE?

Con clases raras medimos el ranking con PR-AUC

Precision

De las alertas que generé, ¿cuántas eran fraude de verdad?

Recall

De todos los fraudes reales, ¿cuántos logré atrapar?

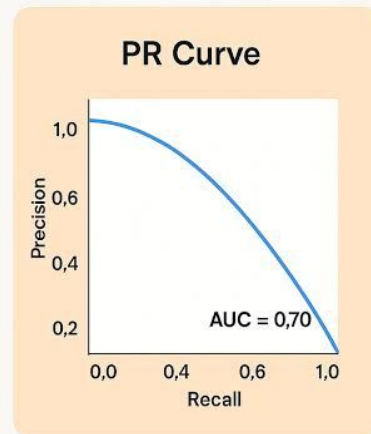
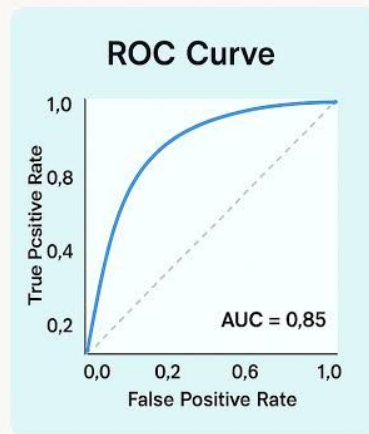
Métrica	Qué pasa con 0,17 % de fraude
Accuracy	El modelo trivial saca 99,8 %. No sirve.
ROC-AUC	Se ve excelente casi siempre: 1 % de falsas alarmas son ~2 800 transacciones.
PR-AUC	Se enfoca en la clase rara. El azar da $\approx 0,0017$ (la prevalencia), no 0,5.

PR-AUC (average precision) resume la curva precision–recall para todos los umbrales posibles: mide qué tan bien el modelo ordena los fraudes por encima de lo normal.

ROC VS PR-AUC

Ambas métricas evalúan qué tan bien separa un modelo dos clases (positivo y negativo) a lo largo de todos los umbrales de decisión posibles, pero se diferencian en cómo tratan a la clase mayoritaria (negativa)

ROC vs PR-AUC



ROC VS PR-AUC

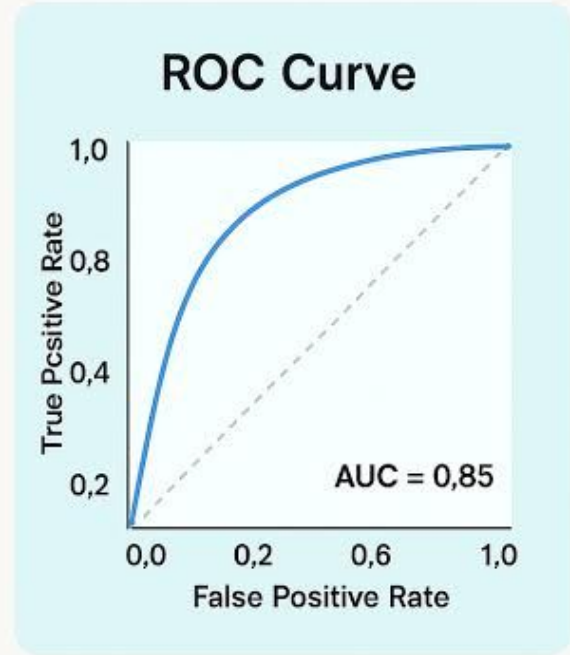
1. Curva ROC (Receiver Operating Characteristic)

Compara qué porcentaje de positivos encuentras frente a cuántos negativos marcas por error.

Eje Y (Recall / True Positive Rate): de los que realmente eran positivos, ¿cuántos atrapó?

Eje X (False Positive Rate): de los que eran sanos/normales, ¿a cuántos alarmó por error?

Línea base (azar): Un modelo que adivine al azar siempre da un AUC de 0.50



ROC VS PR-AUC

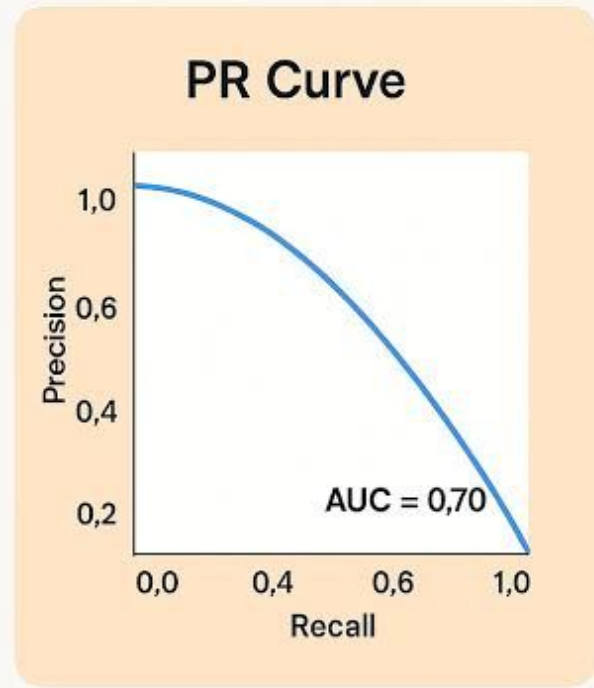
2. Curva PR (Precision-Recall)

Evalúa la calidad de las alertas positivas sin importar cuántos casos negativos existan en total.

Eje Y (Precisión): cuando el modelo dice "positivo", ¿qué tan confiable es?

Eje X (Recall): cobertura total de la clase positiva.

Línea base (azar): No es 0.50; equivale a la proporción de positivos en el dataset (por ejemplo, si solo el 1% es positivo, el azar es 0.01).



Tres estrategias para el desbalance



Sin intervención

El modelo ve los datos tal cual.

Probabilidades realistas (bajas para fraude). Con umbral 0,5 el recall suele ser bajo.



Pesos de clase

Cada error en un fraude cuesta ~577 veces más (scale_pos_weight).

Probabilidades infladas: con 0,5 sube el recall y baja la precisión.



SMOTE

Crea fraudes sintéticos interpolando entre un fraude y sus vecinos.

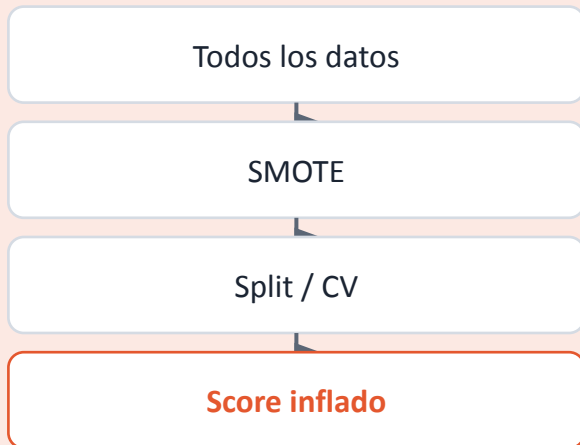
Más ejemplos positivos, pero artificiales. Solo dentro de cada fold.

Clave para la demo: pesos y SMOTE cambian sobre todo la escala de las probabilidades, no necesariamente el orden. Por eso pueden no mejorar el PR-AUC, y no es raro que gane el modelo sin intervención.

SMOTE sin fuga de información



Incorrecto



Los sintéticos se crean con fraudes que después caen en validación: el modelo ya "vio" el examen.



Correcto: dentro del pipeline

En cada fold:

Bloque de entrenamiento

→ escalar → SMOTE → modelo

Bloque de validación

→ intacto, solo predecir

```
ImbPipeline([
    ("scaler", StandardScaler()),
    ("smote", SMOTE(sampling_strategy=0.10)),
    ("model", XGBClassifier(...))])
```

EL PROBLEMA

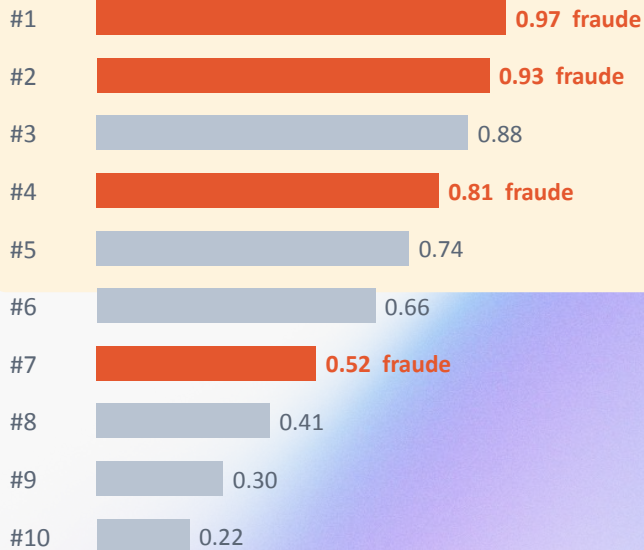
La Capacidad Operativa

-  Volumen masivo: Millones de predicciones diarias.
-  Recursos finitos: Equipo humano de revisión limitado.
-  Filtro estricto: Imposible revisar cada alerta generada.



Recall@k: evaluar como se va a usar el modelo

top-k = 5 alertas que el equipo puede revisar hoy



Transacciones ordenadas por score del modelo

Recall@k

fraudes dentro del top-k $\div$ total de fraudes

Ejemplo: 3 de 4 = 75 %

Precision@k

fraudes dentro del top-k $\div$ k

Ejemplo: 3 de 5 = 60 %

¿Por qué? En producción la capacidad es fija: el equipo revisa k casos al día. PR-AUC sirve para elegir modelo; Recall@k responde "¿cuánto fraude atrapamos con la capacidad que tenemos?"

LA SOLUCIÓN

¿Qué es Recall@k?

$$\frac{\text{Fraudes reales dentro del Top-k}}{\text{Total de fraudes reales}}$$

Mide qué porcentaje de la clase positiva logramos capturar revisando únicamente las primeras **k** posiciones del ranking.

PARA LLEVARSE

Tres ideas

1

Boosting = correcciones pequeñas y controladas.

learning_rate, profundidad, regularización y subsampling son las riendas.

2

Optuna optimiza la función que tú diseñas.

Primero una evaluación confiable (CV + PR-AUC); después la búsqueda bayesiana.

3

Evalúa como se va a usar el modelo.

PR-AUC para escoger, umbral y Recall@k para desplegar, test una sola vez.

Preguntas de
entrevista que
puedes contestar

1. EVALUACIÓN (META / STRIPE)

Tienes un modelo de detección de fraudes donde la prevalencia de la clase positiva es del 0.17%. Tu modelo arroja un Accuracy del 99.8% y un ROC-AUC excelente. Sin embargo, el equipo de operaciones dice que el modelo es inútil. ¿Por qué ocurre esto y qué métricas usarías en su lugar?



Respuesta: El Accuracy es engañoso porque un modelo trivial que predice siempre "no fraude" acertará el 99.8% de las veces. El ROC-AUC también miente en desbalances extremos porque una tasa de falsos positivos del 1% puede equivaler a miles de falsas alarmas (ej. 2800 transacciones). La métrica correcta es el PR-AUC, que evalúa cómo el modelo ordena los fraudes por encima de lo normal, y el Recall@k, que mide cuánto fraude real se atrapa dentro de la capacidad operativa (k) que el equipo humano puede revisar diariamente.

Decides usar SMOTE para balancear tu dataset. ¿Aplicas SMOTE antes de hacer la partición de validación cruzada (Cross-Validation) o durante? Explica el porqué.



SMOTE debe aplicarse estrictamente durante la validación cruzada, dentro del pipeline y solo sobre el bloque de entrenamiento de cada fold. Si se aplica a todo el dataset antes de separar, ocurre una "fuga de información" (leakage) porque los datos sintéticos se crean usando fraudes que posteriormente caerán en el set de validación; esto provoca que el modelo evalúe ejemplos que ya "vio" implícitamente, inflando el score.

¿Cuáles son las diferencias principales en la forma en que XGBoost, LightGBM y CatBoost construyen sus árboles y manejan los datos?



XGBoost: Crece los árboles por niveles (level-wise), creando árboles balanceados y evalúa cortes usando un puntaje de similitud y regularización explícita.

LightGBM: Crece por hojas (leaf-wise), dividiendo siempre la hoja que más reduce el error, lo que genera árboles asimétricos y profundos. Usa GOSS para concentrarse en los errores grandes y es ideal para millones de filas.

CatBoost: Utiliza árboles simétricos (oblivious) donde se hace la misma pregunta en todo un nivel, y resuelve el problema de variables categóricas usando "ordered boosting", barajando las filas para que la codificación solo use información del pasado, evitando fugas.

Típica en FAANG

¿Cuál es la diferencia fundamental entre cómo aprende un modelo de Bagging (como Random Forest) frente a un modelo de Gradient Boosting?



En Random Forest (Bagging), se construyen aprendices independientes que luego se promedian, asumiendo que un gran número de modelos razonables no cometerán el mismo error, lo que reduce la varianza. En Gradient Boosting, se crea una secuencia de árboles donde el objetivo de cada nuevo árbol es aprender de los errores (residuos) del anterior, acercándose poco a poco a la solución mediante una tasa de aprendizaje para minimizar el sesgo sin sobreajustar.

Típica en Roles de ML Ops

Tienes un modelo de Deep Learning o Boosting que tarda 5 minutos en entrenar por cada combinación de hiperparámetros. ¿Por qué utilizarías Optimización Bayesiana sobre Random Search?



Random Search no tiene memoria; prueba combinaciones al azar y no aprende de los intentos desastrosos. La Optimización Bayesiana (mediante algoritmos como TPE) utiliza los resultados anteriores para construir un modelo probabilístico sustituto. Separa los historiales en un grupo "bueno" (top 10%) y uno "malo", calculando sugerencias que maximizan la probabilidad de éxito (explotación) o que reducen la incertidumbre (exploración), ahorrando horas de cómputo inútil. Además, herramientas como Optuna aplican "pruning", cancelando anticipadamente un entrenamiento (trial) si su rendimiento inicial es peor que la mediana de intentos anteriores.

GRACIAS